# Implement face recognition using a CNN on a custom dataset of face images

To implement a CNN-based face recognition system using a prepared face-image dataset.

Algorithm
1. Import NumPy, TensorFlow and the required libraries.
2. Load the Olivetti face dataset containing grayscale face images.
3. Extract the face images and their corresponding person labels.
4. Normalize the image pixel values.
5. Reshape the images to include a grayscale channel.
6. Divide the dataset into training and testing portions.
7. Create a CNN model with convolutional and max-pooling layers.
8. Flatten the extracted image features.

9. Add a Dense layer for feature learning.
10. Add a softmax output layer containing 40 person classes.
11. Compile and train the CNN model.
12. Evaluate the model and predict the identity class of a test image.

Program

```
import numpy as np
import tensorflow as tf
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from tensorflow.keras import layers, models
tf.keras.utils.set_random_seed(18)
# Load face dataset
faces = fetch_olivetti_faces(
shuffle=True,
random_state=18
)
X = faces.images.astype("float32")
X = X[..., None]
y = faces.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
X, y,
test_size=0.25,
random_state=18,
stratify=y
)
# CNN model
model = models.Sequential([
layers.Input(shape=(64, 64, 1)),
layers.Conv2D(32, 3, activation='relu'),
layers.MaxPooling2D(),
layers.Conv2D(64, 3, activation='relu'),
layers.MaxPooling2D(),
layers.Flatten(),

layers.Dense(128, activation='relu'),
layers.Dropout(0.3),
layers.Dense(40, activation='softmax')
])
# Compile
model.compile(
optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy']
)
# Train
model.fit(
X_train, y_train,
epochs=15,
batch_size=16,
validation_split=0.1
)
# Evaluate
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
```

```
# Prediction
prediction = model.predict(X_test[:1], verbose=0).argmax(axis=1)[0]
print("Test Accuracy:", accuracy * 100, "%")
print("Actual Person Class:", y_test[0])
print("Predicted Person Class:", prediction)
```

```
Output of Program
Epoch 1/15
Epoch 2/15
...
Epoch 15/15
Test Accuracy: Training-dependent value
Actual Person Class: ...
Predicted Person Class: ...
```

Result

Thus, a CNN-based face recognition system was successfully implemented using a faceimage dataset. The model lea
images.